

Теория параллелизма

Отчет

Решение уравнения теплопроводности с использованием метода Якоби

Выполнил Спиркин Алексей Александрович, гр. 23930

30.05.2025

Цель работы: применить инструменты CUDA для распараллеливания метода Якоби в решении уравнения теплопроводности с заданными стартовыми условиями.

Компиляторы:

`nvc++ 23.11-0 64-bit` (с поддержкой CUDA Toolkit 12.3, не имеющимся на сервере).

Визуализатор параллельного кода:

`nsys-ui` (Nsight Systems GUI).

Инструмент для измерения времени работы:

`chrono`.

Выполнение на CPU (по факту добавления критических оптимизаций)

CPU-onecore

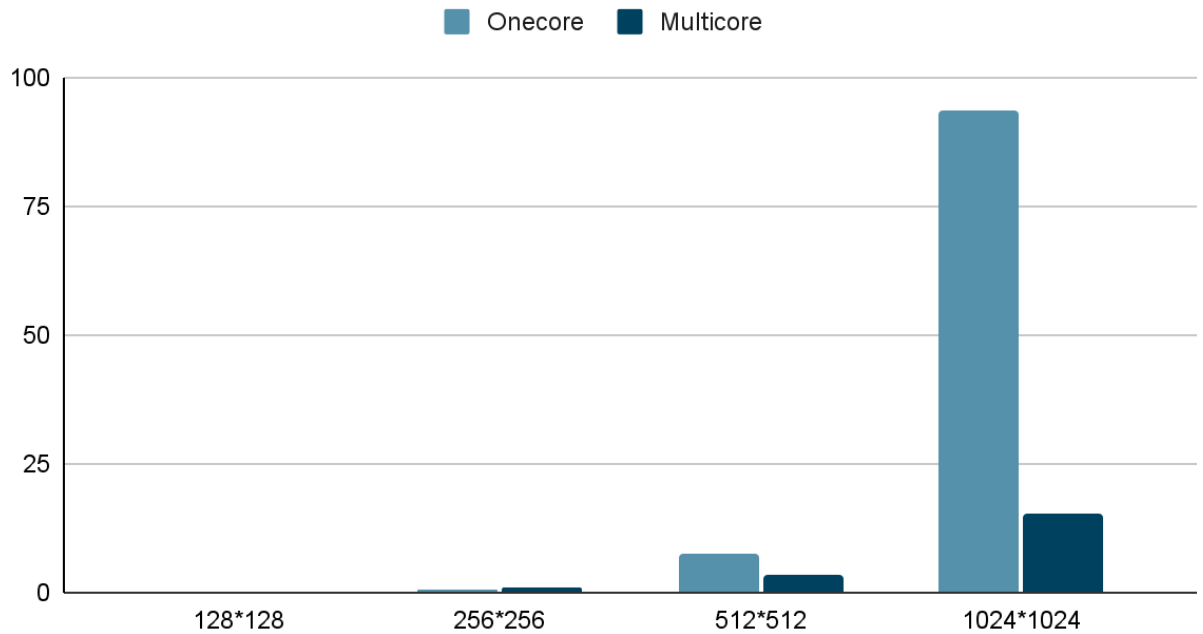
Размер сетки	Время выполнения, сек	Точность	Количество итераций
128*128	1.98145	9.9998e-07	30074
256*256	7.00646	9.9993e-07	102885
512*512	73.85040	9.99984e-07	339599
1024*1024	937.38000	1.36929e-06	1000000

CPU-multicore

Размер сетки	Время выполнения, сек	Точность	Количество итераций
128*128	2.45321	9.9998e-07	30074
256*256	8.48421	9.9993e-07	102885
512*512	35.13850	9.99984e-07	339599
1024*1024	155.15400	1.36929e-06	1000000

Диаграмма сравнения времени работы CPU-onecore и CPU-multicore

Сравнение onecore и multicore



Выполнение на GPU

Измерения на матрице 512*512 с указанием оптимизаций для ускорения (на 10e+6 итерациях)

№	Время выполнения, сек	Точность	Описание решения
1	213.326	9.99984e-07	Используется OpenACC без применения оптимизаций.
2	203.876	9.99984e-07	OpenACC, убран вывод промежуточного результата по окончании каждой итерации.
3	116.794	9.99984e-07	OpenACC, изменена инициализация матриц с <code>std::vector</code> на <code>aligned_alloc (64, N*N*8)</code>
4	23.858	9.99984e-07	OpenACC/cuBLAS, реализован поиск максимальной ошибки через <code>cublasIdamax</code> по матрице расхождений <code>Anew</code> от <code>A</code> .
5	10.7431	9.99984e-07	Реализация на CUDA.

Визуализация в nsys-ui (10000 итераций)

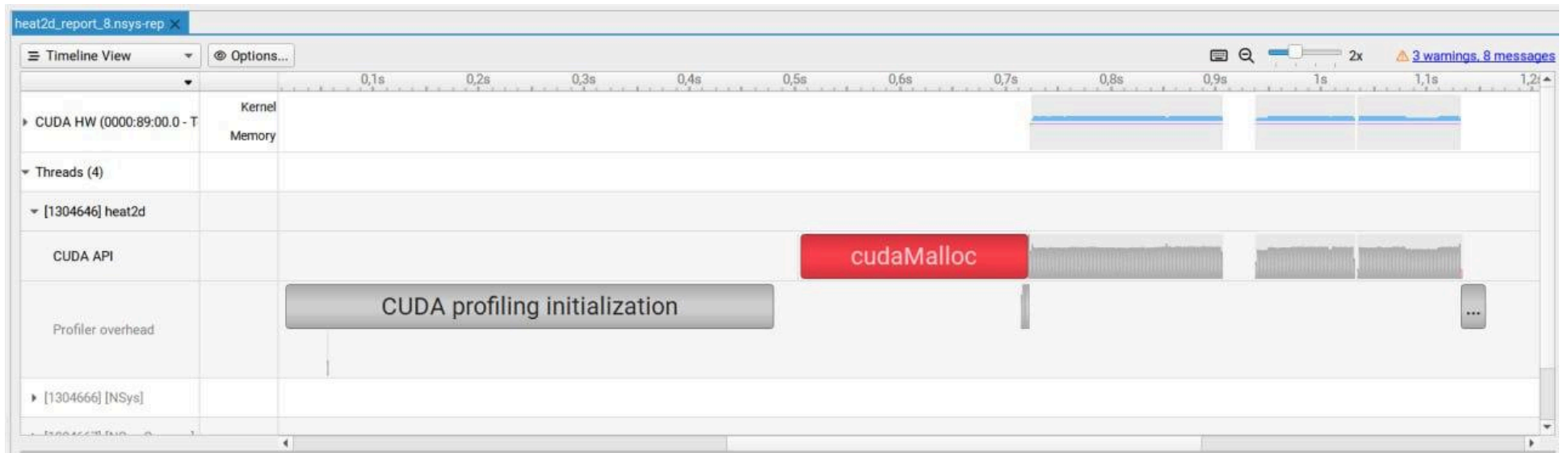
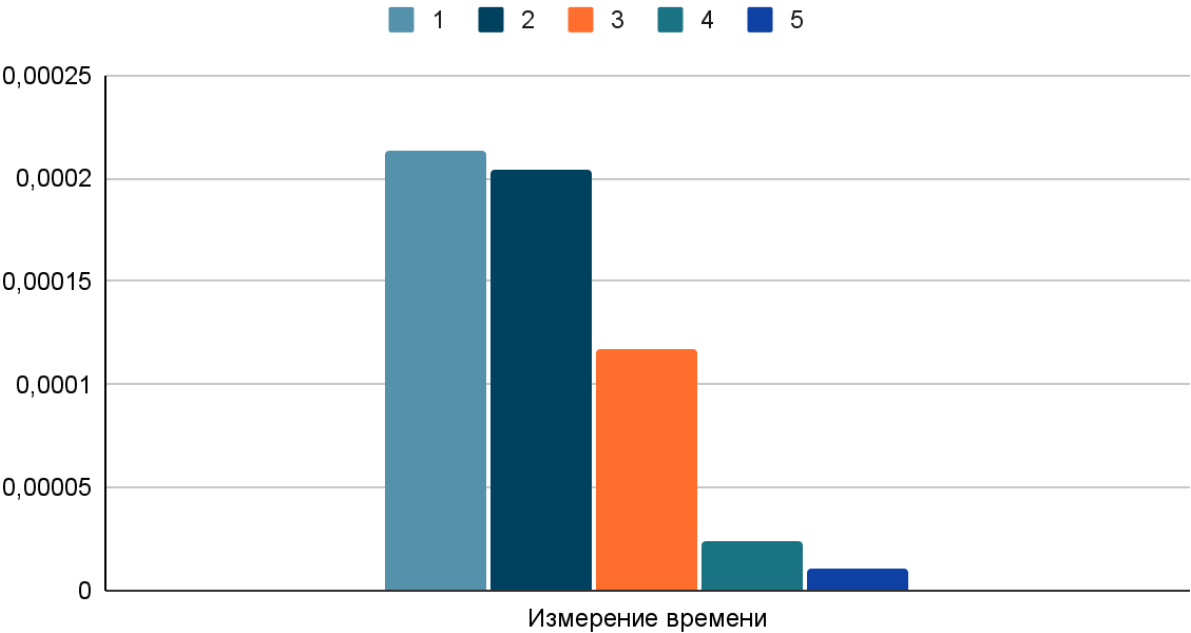


Диаграмма оптимизации

Измерение времени по оптимизации

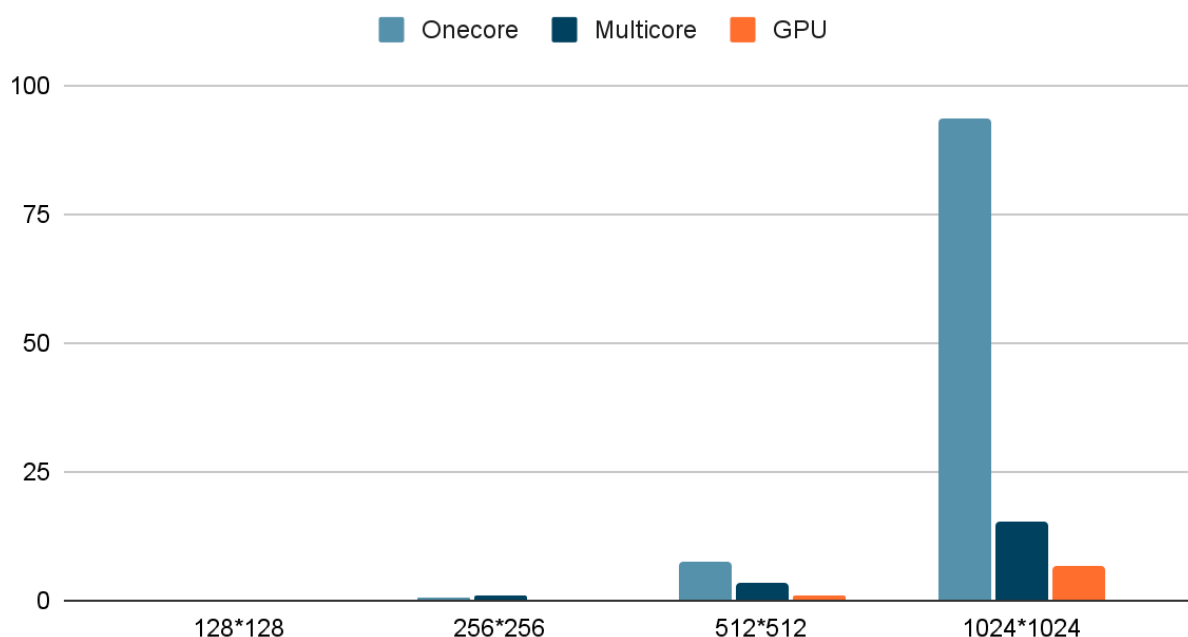


Измерения оптимизированного варианта на GPU

Размер сетки	Время выполнения, сек	Точность	Количество итераций
128*128	0.802595	9.9998e-07	30074
256*256	2.83189	9.9993e-07	102885
512*512	10.7431	9.99984e-07	339599
1024*1024	68.3462	1.36929e-06	1000000

Диаграмма сравнения времени работы CPU-onecore, CPU-multicore, GPU-optimized для разных размеров матриц

Сравнение onecore, multicore и GPU



Вывод

Использование нескольких ядер CPU позволяет значительно увеличить производительность, однако при использовании GPU, несмотря на удовлетворительные первичные показатели по времени, возможно многократное увеличение скорости вычислений.

Использование инструментов интерфейса CUDA позволяет полностью реализовывать память и вычислительную производительность GPU для сокращения времени подсчета результата практически без погрешностей на дополнительную обработку.

И если ранее предполагалось, что несоответствие версии CUDA Toolkit к версии nvcc++ препятствует оптимизации исполняемого алгоритма, то сейчас шансы этого либо совершенно незначительны, либо исключены.